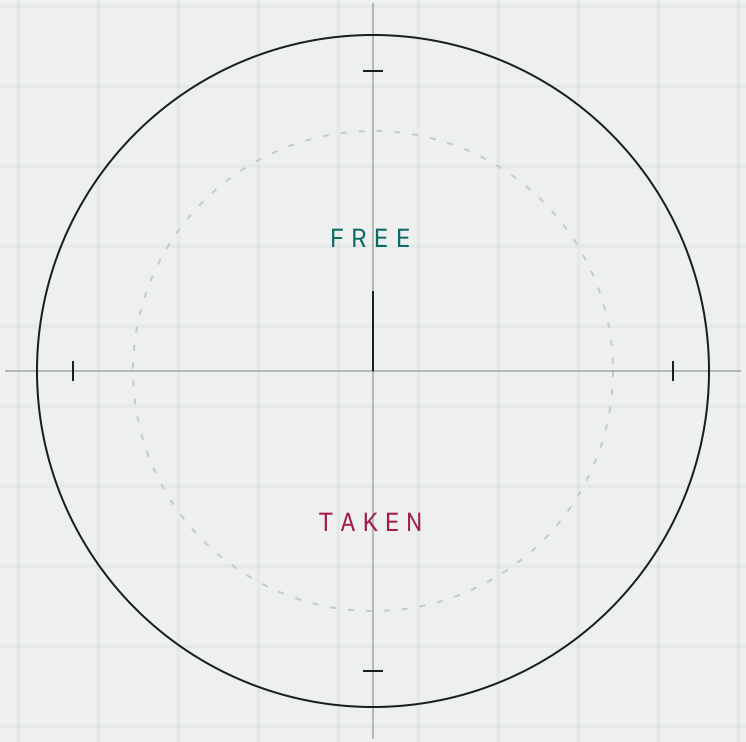
PROJECT

Shared Laboratory Equipment Booking System

+ Arjun Tandon
+ Ansh Jaiswal
+ Priyanshu Sharma
+ Lakshit Verma



Sharing instruments without a register.

01 / CLAIM

Two students can book the same microscope for the same afternoon.

02 / TRACE

Once an instrument leaves the shelf, nobody can say who has it.

03 / COST

Late returns are remembered, not recorded, so no fine can be charged.

One console app that holds the register.

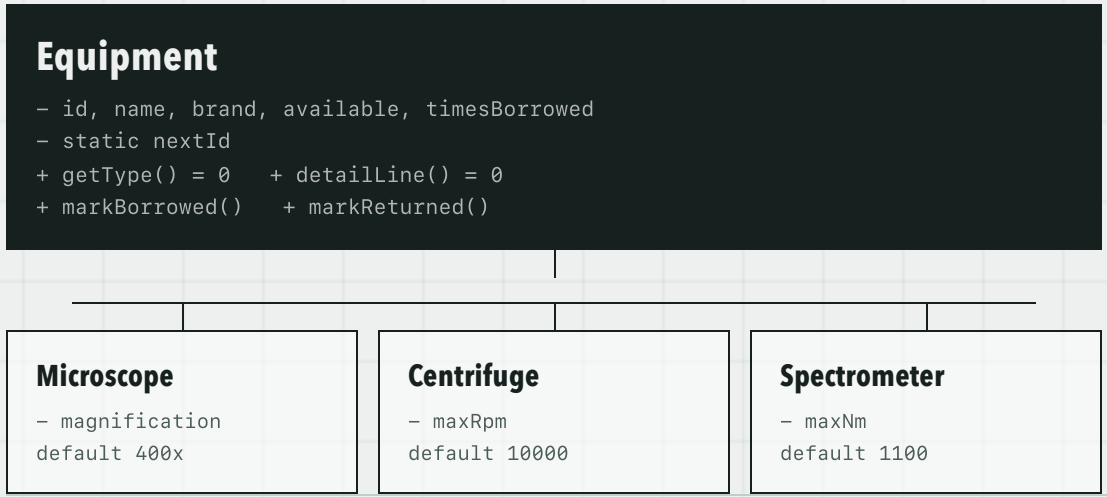
A C++17 build with no database. Gear, users and bookings live in memory, and every rule sits in one class. Borrowing an item marks it taken, returning it closes the record and prices the delay.

Two hierarchies carry the data. One manager class enforces the rules.

```
Equipment.h  User.h  Booking.h
BookingSystem.h  Types.h  main.cpp
```

Two trees, same shape.

EQUIPMENT



USER



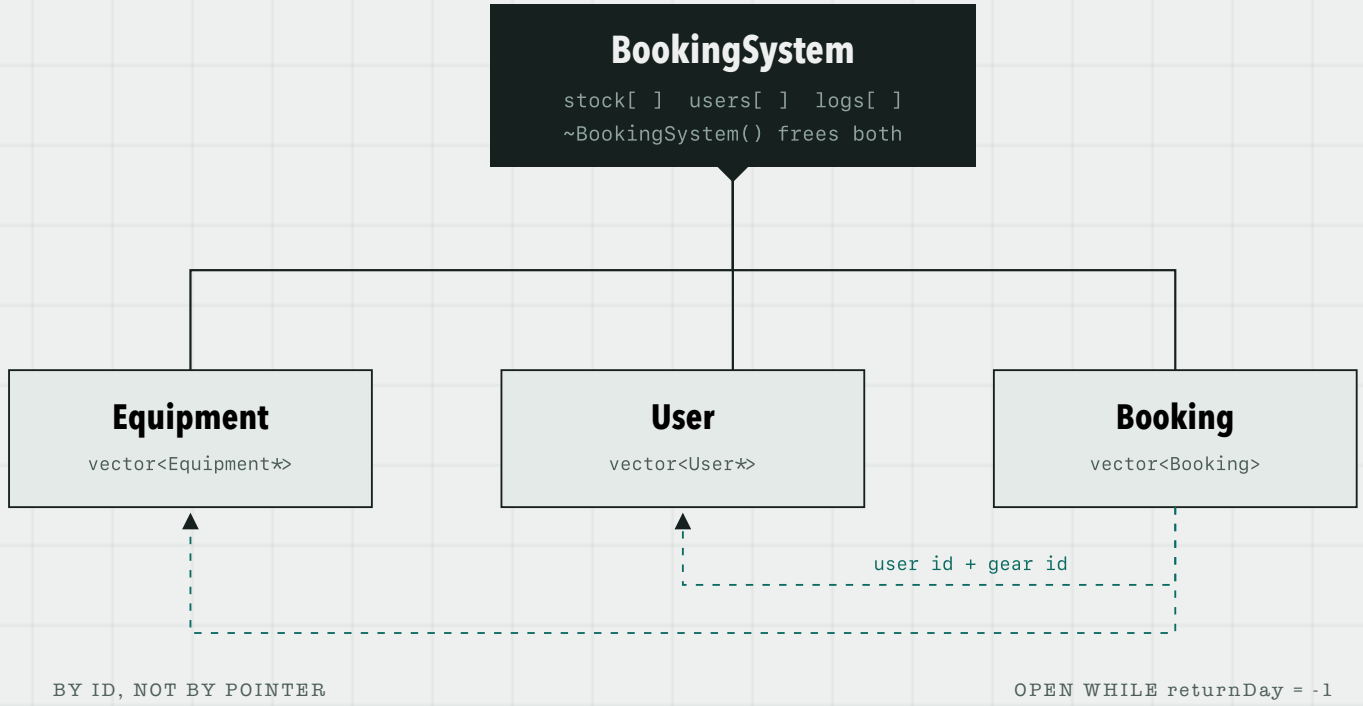
NO BASE OBJECT

Every child must answer `getType()` and `detailLine()`. The base has no body for either, so `Equipment e;` cannot compile.

YEAR CHANGES THE QUOTA

Student computes `max = 1 + studyYear`. Year 1 holds 2, year 2 holds 3, year 3 holds 4.

Composition holds the lists. Association links a record.



COMPOSITION

BookingSystem owns the three lists and frees every pointer in its destructor. Kill the manager and the register dies with it.

ASSOCIATION

A Booking stores a user id and a gear id. It does not own either object, so a deleted item cannot leave a broken pointer behind.

DEPENDENCY

main talks only to BookingSystem. It never reaches into the vectors, so every rule passes through one door.

`borrow()` checks user, gear and quota, then writes a Booking and calls markBorrowed(). One door for every rule.

Every concept has a call site.

CONCEPT	WHERE	WHAT IT DOES IN THIS BUILD
Encapsulation	`Equipment.h` `User.h / Booking.h`	All state is private. An item can only leave the shelf through `markBorrowed()`, so no caller can set `available` by hand.
Inheritance	`Equipment.h` `User.h`	Three gear types and three roles. Each child adds only its own fields and inherits the rest.
Polymorphism	`detailLine()` `getRole() / getMaxItems()`	`Equipment* e; e→detailLine()` prints the child line. The list code never checks the type.
Abstraction	`pure virtual getType()` `detailLine()`	The base says every item must describe itself. It does not say how. No base object can exist.
Overloading	`constructors` `search() / operator<<`	`Microscope(n,b)` and `Microscope(n,b,mag)`. `search(word)` and `search(id)`.
Statics and structs	`nextId counters` `Result / GearSpec`	One shared id source per class. Structs carry plain results and build specs across calls.

Quota by role, fine by day.

ROLE	ITEMS	LOAN	FINE / DAY
Student, year 1	2	7	10
Student, year 2	3	7	10
Student, year 3	4	7	10
Faculty	5	14	5
LabAdmin	10	30	0

FINE

```
fine = (returnDay − dueDay)
       x finePerDay

0 when on time
```

Student leaves on day 1, due day 8, returns day 9. One day late at 10 per day is 10.

Quota comes from `getMaxItems()`. The child decides, the manager obeys.

ITEM ALREADY OUT

Borrow fails and names the holder id.

QUOTA REACHED

Borrow fails, prints held count and the role max.

REMOVE WHILE OUT

Admin remove refuses until the item is back.

What we will run in class.

01 Login as user 1, a year 1 student.

02 List gear. Three instruments, all free.

03 Borrow 101, then try 101 again.
`TAKEN by user 1`

04 Borrow 102, then try 103.
`quota 2 of 2 reached`

05 Jump to day 9, return 101.
`late, fine 10`

06 Search "Micro", then search id 102.
`one call name, two forms`

07 Add gear as admin with spec 0.
`short constructor runs`

08 Show the hierarchy and the diagram.

```
$ ./labbook
> 1 login 1                    Priyanshu (Student year 1 max=2)
> 3 borrow 101                 OK, due day 8
> 3 borrow 101                 FAIL: TAKEN by user 1
> 3 borrow 102                 OK
> 3 borrow 103                 FAIL: holds 2, max is 2
> 0 (x8) then 4 return 101     LATE: day 9, fine 10
> 6 search Micro               1 row
> 7 search 102                 1 row, same function name
```

One command, no dependencies.

```
# build
g++ -std=c++17 -Wall main.cpp -o labbook

# run
./labbook
```

Five headers and one main file. Standard library only, and the build is warning free.

IN MEMORY ONLY

Gear, users and bookings reset when the process ends. No database was allowed, so none is used.

MANUAL DAY COUNTER

Day moves with one menu key. Dates are relative, not calendar dates.

NEXT

Save the register to a file, and swap the day counter for real dates.

CODE

github.com/vee1e/lab-booking-system

Questions